

SwarmOps Cloud

Software Architecture Document

September 2026

Abstract

This document explains how SwarmOps Cloud is built and why. It follows the arc42 template: goals and constraints, context, solution strategy, building blocks, runtime scenarios, deployment, cross-cutting concepts, decisions, quality scenarios, and risks. The structural views are drawn with the C4 model and given in full in the separate C4 Architecture Model; this document refers to them and adds the behaviour, the rationale and the trade-offs. The central architectural idea is that the command queue and the wallet ledger share one relational database. That makes paying for an application and requesting its deployment a single local transaction.

Contents

1	Introduction and goals	2
1.1	Requirements overview	2
1.2	Quality goals	2
1.3	Stakeholders	2
2	Constraints	3
2.1	Technical constraints	3
2.2	Organisational constraints	3
2.3	Conventions	3
3	Context and scope	3
3.1	Business context	3
3.2	Technical context	3
4	Solution strategy	4
5	Building block view	5
5.1	Level 1: containers	5
5.2	Level 2: Core's components	5
5.3	Mapping of the former snapshot files to tables	5
6	Runtime view	6
6.1	Confirming an order	6
6.2	Command execution and retries	7
6.3	Hourly billing, suspension and resumption	7
6.4	Storefront sign-in and request protection	7
6.5	Start-up and schema migration	7
7	Deployment view	7
8	Cross-cutting concepts	9
8.1	Persistence and transactions	9
8.2	Portability between MariaDB and MySQL	9
8.3	Money	9
8.4	Security	10
8.5	Concurrency and ownership	10
8.6	Error handling	10
8.7	Internationalisation	10
8.8	Testing	11
9	Architecture decisions	11
10	Quality requirements	11
10.1	Quality scenarios	11
11	Risks and technical debt	12
12	Glossary	13

List of Figures

1	Sequence of an order confirmation	6
-------------------	---	---

2	Sequence of hourly billing	8
---	--------------------------------------	---

List of Tables

1	Quality goals	2
3	Technical context	4
4	Solution strategy	4
5	Packages	5
6	Where each former store's data now lives	5
7	Deployment configuration	9
8	Error mapping	10
9	Architecture decisions	11
10	Quality scenarios	12
11	Risks and technical debt	12

1 Introduction and goals

1.1 Requirements overview

SwarmOps Cloud sells container hosting. Customers top up a wallet and order plans; administrators confirm orders; the platform deploys each application to Docker Swarm and bills it hourly. The Software Requirements Specification states the requirements in full.

1.2 Quality goals

Table 1: Quality goals

Priority	Quality goal	Motivation
1	Integrity of money	A balance must never be negative or drift from its ledger, and no request may charge or credit twice.
2	Atomic provisioning	A customer must never pay for a deployment that was not requested, nor receive one that was not paid for.
3	Security	Customers must be isolated from each other and from administration; secrets must stay encrypted.
4	Portability	The system must run on both MariaDB 11.4 and MySQL 8.4.
5	Operability	Failures must be visible, explained and recoverable by an administrator, and every change must be audited.

1.3 Stakeholders

Stakeholder	Expectation
Reviewer	A complete, correct system using SQL, with an administration panel and a storefront, documented thoroughly.
Customers	A simple, trustworthy way to run an image and pay only for what runs.
Administrators	Control over orders, prices, money and the platform, with evidence for every decision.
Developers	Clear module boundaries, tests on both engines, and diagrams that match the code.

2 Constraints

2.1 Technical constraints

- **TC-1** Go for the server, React with the existing component kit for the interfaces.
- **TC-2** MySQL or MariaDB for all controller state, restricted to features both engines share.
- **TC-3** The existing public interfaces of the SwarmOps stores are preserved.
- **TC-4** The data-encryption key is never stored in the database.
- **TC-5** One active core acts on the cluster at a time.

2.2 Organisational constraints

- **OC-1** One developer, within a single development cycle.
- **OC-2** Everything must be demonstrable on a laptop: a single-node Swarm, local databases, a loopback machine agent.
- **OC-3** No real payment provider may be contacted.

2.3 Conventions

- Go code follows the repository's existing style and is checked with `go vet`.
- The console follows its information-architecture rules, enforced by `operator-workflows.test.mjs`. There is one navigation registry, every page has a route, destructive actions need a typed confirmation phrase, and dates and money are formatted in one module.
- Money is stored as whole rials in `BIGINT` columns and never as floating-point numbers.

3 Context and scope

3.1 Business context

Figure 1 of the C4 Architecture Model shows the business context. Customers and administrators use SwarmOps Cloud. It deploys to a Docker Swarm cluster, which pulls images from registries and obtains certificates from an ACME authority.

3.2 Technical context

Table 3: Technical context

Channel	Protocol and format	Authentication
Browser → Core (storefront)	HTTPS, JSON, /api/store/v1	Session cookie <code>swarmops_store_session</code> and <code>X-CSRF-Token</code> header
Browser → Core (console)	HTTPS, JSON, /api/v1	Operator session cookie and CSRF token
Core → database	MySQL protocol over TCP	DSN from a protected file
Core → machine agent	HTTPS, JSON	Pinned TLS certificate and API key
Agent → Docker Engine	Docker Engine API over a local socket	Local socket permissions
Visitor → gateway → application	HTTP/HTTPS through Traefik	Application-defined

4 Solution strategy

Table 4: Solution strategy

Goal or constraint	Approach	Where
Integrity of money	An append-only ledger is the source of truth; a cached balance is locked and updated with each row; a <code>CHECK</code> constraint forbids a negative balance; idempotency keys are unique per user.	Section 8.3; ADR-0008
Atomic provisioning	The deployment command is inserted by <code>SubmitInTx</code> inside the confirmation transaction (transactional outbox).	Section 6.1
One owner for background work	Only the active core runs the command worker and billing. Singleton rows are locked with <code>FOR UPDATE</code> .	Section 8.5
Portability	Shared SQL subset; the full test suite runs on both engines.	Section 8.2
Preserve SwarmOps	Stores keep their interfaces; only constructors change.	ADR-0008
Billing like Heroku and Liara	Hourly charges, one row per project per hour, capped monthly, with suspension and resumption.	Section 6.3
Security	Bcrypt, <code>HttpOnly</code> <code>SameSite=Strict</code> cookies, CSRF tokens, rate-limited sign-in, sealed columns with associated data, and audit.	Section 8.4

5 Building block view

5.1 Level 1: containers

The system consists of the storefront, the administration console, SwarmOps Core and the controller database. The machine agent, gateway and applications run in the cluster. See Figure 2 of the C4 Architecture Model.

5.2 Level 2: Core's components

Figures 3 and 4 of the C4 Architecture Model show Core and the commerce service. The packages that matter to this project are:

Table 5: Packages

Package	Responsibility
internal/sqlstore	Pool, <code>WithTx</code> with retries, migrations, sealed columns, error classification.
internal/sqlstore/migrations	0001_audit.sql to 0009_plan_text.sql.
internal/cloud	Commerce business rules.
internal/queue	Command store (Store) and worker (Worker).
internal/audit, internal/ops, internal/source, internal/remote, internal/coretopology, internal/agentpull	The SwarmOps stores, now on SQL, and the control plane.
api/http	HTTP routing, authentication, the storefront and commerce APIs.
cmd/api	Process wiring, <code>migrate-state</code> , the billing loop.
web/src/store	Storefront.
web/src/screens/sales	Sales area of the console.

5.3 Mapping of the former snapshot files to tables

Table 6: Where each former store's data now lives

Former snapshot	Tables
audit.sealed	audit_events, audit_event_details
applications.sealed	applications, application_env, application_health_command, application_databases, application_database_env, application_outcomes
database-credentials.sealed	database_credentials, application_database_credentials
source-connections.sealed, source-settings.sealed	source_connections, source_settings, source_private_hosts
servers.sealed, server-keys.sealed	servers, server_agent_events, server_keys
core-topology.sealed	core_authority, core_members, core_handoffs

Former snapshot	Tables
agent-pull-registry.sealed	agent_ca, agents,
traefik-routing.sealed	agent_enrollment_tokens, agent_claims
	routing_clusters, routes, route_hosts,
	dependency_bindings,
	service_route_declarations,
	routing_domains,
	dns_credential_versions, dns_records,
	route_certificates,
	route_certificate_domains,
	route_runtime,
	route_runtime_entry_points,
	route_runtime_errors
commands/commands.sealed	commands, command_payloads,
	command_events, command_outputs,
	command_locks

6 Runtime view

6.1 Confirming an order

Figure 1 shows the confirmation and what follows it.

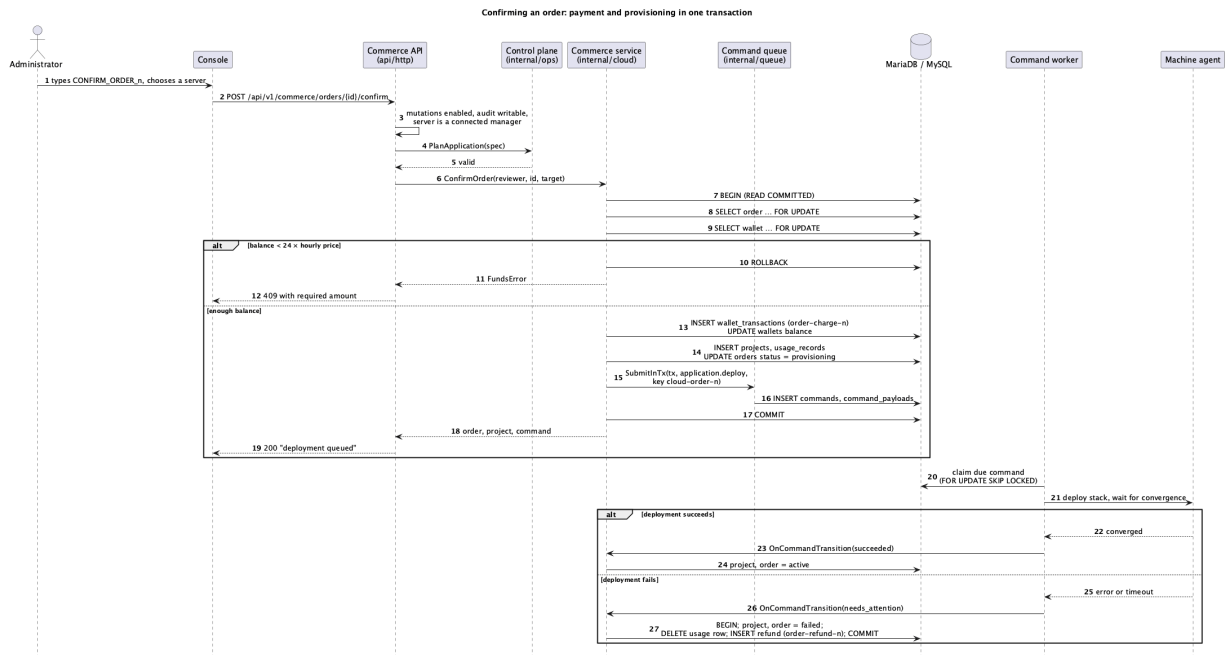


Figure 1: Sequence of an order confirmation

Every check that can fail without cost runs before the transaction opens: whether mutations are enabled, whether the audit ledger and command store can be written, whether the server is a connected manager, and whether the application passes planning. Inside the transaction, `lockPendingOrder` locks the order row and checks it is still pending, and only then is the wallet row locked. Every path that changes money follows the same rule: lock its own business row first, then the wallet row last. That row is the order when confirming, the project when refunding, and the new usage row when billing. Two such transactions may wait for the same wallet but do not

hold each other's rows; if the server aborts one anyway, `WithTx` repeats it. The charge uses the idempotency key `order-charge-<id>` and the command uses `cloud-order-<id>`, so a repeated confirmation neither charges nor queues twice.

6.2 Command execution and retries

The worker polls every 250 milliseconds. `ClaimDue` claims nothing while another command is running. Otherwise it locks the oldest due command, ordered by `created_at` and `seq`, with `FOR UPDATE SKIP LOCKED`. The worker executes it within a timeout: ten minutes for ordinary actions such as `application.deploy`, 35 minutes for image builds, and 50 minutes for source deployments and server readiness.

A failure that is not marked permanent is retried with backoff until `max_attempts` (three for deployments). After that the command moves to `needs_attention` with a failure code, summary and recovery hint. Each transition is recorded as a `command_events` row and passed to `RecordCommandTransition`, which informs the commerce service.

6.3 Hourly billing, suspension and resumption

Figure 2 shows billing.

The unique key on `usage_records(project_id, period_start)` makes a second charge for the same hour impossible, even if two billing runs overlap. The monthly cap is checked against the sum of the month's usage inside the same transaction. When a wallet cannot pay, the charge is rolled back, the project is suspended and a `service.scale` command stops it. A top-up calls `ResumeFundedProjects`. That sets `billing_resumed_at`, so billing restarts from the resumption and never charges the suspended hours.

6.4 Storefront sign-in and request protection

Sign-in creates a session row holding the SHA-256 hash of the session token and a random CSRF token. The browser receives the token only in the `HttpOnly` cookie and the CSRF token in the response body. Every change sends the CSRF token in a header, compared in constant time. After eight failed attempts for a key within fifteen minutes, sign-in is refused. If a change is refused because the page holds an outdated CSRF token (for example after signing in from another tab), the storefront reads the current session once and repeats the change with the same idempotency key.

6.5 Start-up and schema migration

At start-up Core opens the database and runs `Migrate`. It takes the named lock `swarmops_schema_migrations` with `GET_LOCK` (waiting up to 60 seconds), checks that applied versions are contiguous and that their checksums match the embedded files, and applies the missing migrations in order. Only then are the stores constructed. The active core recovers interrupted commands and starts the worker and the five-minute billing loop.

7 Deployment view

Figures 7 and 8 of the C4 Architecture Model show the intended production deployment and the environment used for verification. Production configuration is read from files:

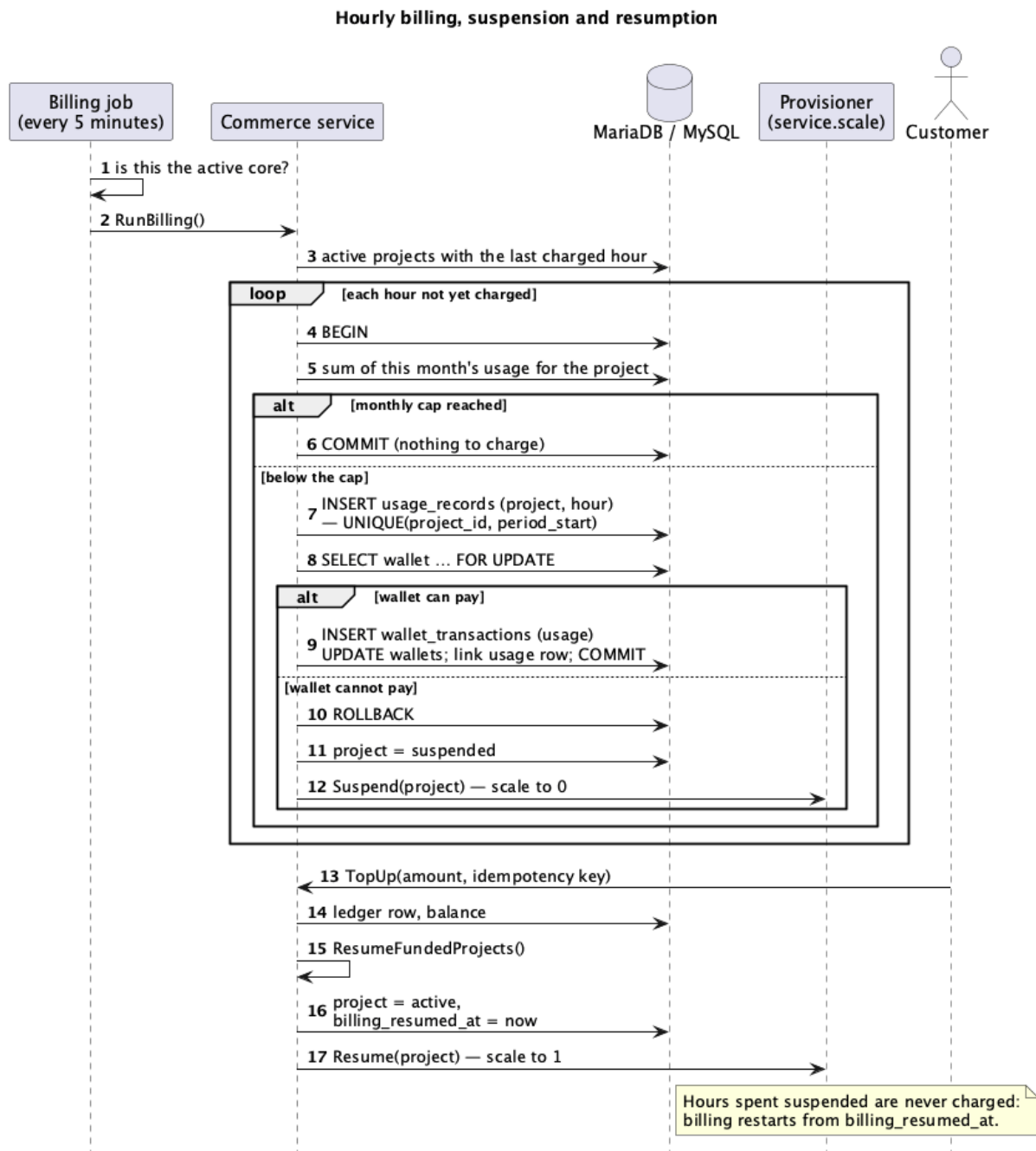


Figure 2: Sequence of hourly billing

Table 7: Deployment configuration

Setting	Purpose
<code>SWARMOPS_DATABASE_DSN_FILE</code>	Protected file holding the DSN.
<code>SWARMOPS_DATABASE_MAX_OPEN_CONNS</code>	Upper bound of the connection pool.
Data-key and session-key files	Encryption of secret columns and signing of operator sessions.
<code>swarmops_database_dsn</code> Swarm secret	The same DSN when Core runs as a Swarm service (<code>deploy/stacks/swarmops.yml</code>).

Backups are taken with `mysqldump --single-transaction`, which gives a consistent InnoDB snapshot without blocking writers. Because secret columns are encrypted with a key that is not in the database, a backup must be stored together with a separately protected copy of that key. The Installation and Operations Guide describes backup, restore and upgrade.

8 Cross-cutting concepts

8.1 Persistence and transactions

All writes that belong together run in `sqlstore.DB.WithTx`, at `READ COMMITTED` isolation. `READ COMMITTED` was chosen over the InnoDB default of `REPEATABLE READ` because the code locks the rows it reads for update explicitly. It avoids `REPEATABLE READ`'s gap locks on range reads, which would serialise unrelated inserts such as usage rows for different projects. A transaction aborted with error 1213 (deadlock) or 1205 (lock-wait timeout) is re-run up to four attempts in total, waiting $\text{attempt}^2 \times 15$ ms between attempts; the whole function is repeated, so partial effects cannot leak.

8.2 Portability between MariaDB and MySQL

Only features present in both engines are used. Two differences affected the implementation:

- **sensitive** is a reserved word in both, so the column is named `is_sensitive`.
- The `CHECK`-violation error number differs (3819 in MySQL, 4025 in MariaDB), so `sqlstore.IsCheckViolation` recognises both.

Datetimes are stored as `DATETIME(6)` in UTC, and imported times are truncated to microseconds so values compare equal on both engines.

8.3 Money

- **Units.** Amounts are integers in rials, shown in tomans.
- **The ledger.** Every change is a `wallet_transactions` row carrying a kind, signed amount, resulting balance, reference and optional idempotency key. `UNIQUE(user_id, idempotency_key)` makes retries safe. The cached `wallets.balance_rial` is updated in the same transaction as the row, under the wallet row lock, and `CHECK (balance_rial >= 0)` forbids overdraft. The console verifies that the balance equals the sum of the ledger.
- **Denormalisation.** Keeping a cached balance is a deliberate step away from strict normal form. It keeps the balance check a single-row read under a lock, instead of a sum over a growing ledger. The ledger remains the authority.
- **Tax.** Prices include tax. The tax in a total T at rate r basis points is $T \cdot r / (10000 + r)$, rounded down, and the invoice's subtotal is T minus that tax.

8.4 Security

- **Customers.** Passwords are stored as bcrypt hashes (cost 12). An unknown email is checked against a dummy hash so its response time matches a wrong password. Sessions are stored as token hashes, never tokens. Cookies are HttpOnly, SameSite=Strict and Secure when configured. Changes require a CSRF token.
- **Isolation.** Every customer query is scoped by the session's user id. Administration routes accept only operator sessions.
- **Secrets.** Secret columns are sealed with AES-GCM. The associated data `swarmops-sql:<table>.<column>` binds each ciphertext to its row, so a value copied elsewhere fails to open.
- **Audit.** Commerce events are written to the audit ledger with the actor:
 - by customers: registrations, top-ups and placed orders;
 - by administrators: order confirmations (with the queued command) and rejections, wallet adjustments, customer status changes, plan changes, invoice issuing and on-demand billing runs.
- **Confirmations.** Costly or destructive console actions require typing a phrase such as `CONFIRM_ORDER_3`.

8.5 Concurrency and ownership

Row locks replace the in-process mutexes the snapshot stores used, so correctness no longer depends on there being one process. Work with a single owner locks a singleton row: `core_authority` for the core topology and its authority epoch, and `agent_ca` for the agent certificate authority. Named commands are serialised by upserting a row in `command_locks`. Only the active core recovers commands, executes them and bills.

8.6 Error handling

Commerce errors are mapped to HTTP responses in one place (`cloudError`):

Table 8: Error mapping

Error	Status	Body
Validation	422	<code>error</code> and the offending <code>field</code>
Insufficient reserve	409	<code>error</code> , <code>balanceRial</code> , <code>requiredRial</code>
Insufficient funds, conflict (for example a taken name)	409	<code>error</code>
Not found	404	<code>error</code>
Invalid credentials, expired session	401	<code>error</code>
Forbidden, invalid CSRF token	403	<code>error</code>
Anything else	500	a generic message and the request id; the cause is logged

8.7 Internationalisation

The storefront keeps English and Persian dictionaries with the same keys; the type system rejects a key missing from either. Choosing Persian switches the component kit to right-to-left layout and the `fa-IR` locale, which formats digits and dates in Persian. Server validation messages are in English only.

8.8 Testing

`sqltest` gives each test a throwaway schema on a real server. Tests cover:

- concurrency: fifty parallel charges against one wallet;
- idempotency;
- atomicity: a confirmation whose command cannot be queued charges nothing;
- billing caps and suspension;
- invoices;
- authorisation over HTTP;
- migrations, including a forced deadlock that `WithTx` must survive.

CI runs the suite on MariaDB and on MySQL. The Test Plan and Report gives the results.

9 Architecture decisions

Table 9: Architecture decisions

Decision	Summary	Record
Controller state in MySQL or MariaDB	All eleven stores move to one relational database with migrations and sealed columns.	ADR-0008 (docs/adr/ADR-0008-sql-controller-stat
Transactional outbox for provisioning	The deployment command is inserted in the confirmation transaction rather than sent after it.	This document, section 6.1
Append-only ledger with cached balance	The ledger is the authority; the balance is a locked cache protected by a CHECK constraint.	Section 8.3
Wallet and administrator confirmation	Prepaid wallet with simulated top-ups; each order is confirmed by an administrator, who chooses the server.	Proposal, section 6
Hourly billing with a monthly cap	One usage row per project per hour; suspension instead of debt.	Section 6.3
Storefront as a second Vite entry	Separate <code>store.html</code> sharing the component kit and served by the same binary.	C4 Architecture Model, section 3

10 Quality requirements

10.1 Quality scenarios

Table 10: Quality scenarios

Quality	Scenario	Expected response	Evidence
Integrity	Fifty concurrent charges of 30,000 rials hit a wallet holding 1,000,000 rials.	Exactly 33 succeed, 17 are refused, the balance is 10,000 and equals the ledger.	<code>TestConcurrentChargesNeverOver</code>
Integrity	A top-up request is sent twice with the same idempotency key.	The wallet is credited once.	<code>TestATopUpIsCreditedOncePerIden</code>
Atomicity	The command queue refuses the deployment during confirmation.	Nothing is charged and the order stays pending.	<code>TestAConfirmationThatCannotQue</code>
Reliability	A deployment fails after the charge.	The order fails and one refund is recorded, even if the outcome is reported twice.	<code>TestTheDeploymentOutcomeActivat</code> orders 1 and 2 in the end-to-end run
Security	A customer requests another customer's order.	404, and nothing is revealed.	HTTP tests in <code>api/http/cloud_api_test.go</code>
Security	A change is sent with an outdated CSRF token.	403; the storefront recovers once with the current token.	End-to-end ticket scenario
Portability	The full test suite runs against each engine.	All tests pass on MariaDB 11.4 and MySQL 8.4.	Test Plan and Report

11 Risks and technical debt

Table 11: Risks and technical debt

Risk or debt	Effect	Possible remedy
The worker runs one command at a time, and a deployment waits for its service to converge.	An image that never becomes healthy blocks all other commands until its ten-minute timeout. This was observed with order 2.	Fail deployments early on repeated health-check failures, or run commands for different servers concurrently.
A failed project keeps its application name.	The customer cannot reorder under the same name.	Release names of failed projects, or let customers delete them.
Billing follows the project's status, not observed running time.	Hours during which the host was down are charged (defect D-10).	Charge only hours in which Swarm reports a running task.
No public default domain.	Customers cannot reach their applications from the internet.	A platform wildcard domain with DNS-01 certificates.

Risk or debt	Effect	Possible remedy
Payments are simulated.	Not usable commercially as is.	Integrate a gateway; the idempotency key is ready for its reference.
Billing runs every five minutes.	An hour's charge can appear up to five minutes after the hour begins.	Acceptable; shorten the interval if needed.
Server messages are English only.	Persian customers see English validation errors.	Return message keys and translate them in the storefront.
The database is a single instance.	It is a single point of failure.	A replicated MariaDB or MySQL deployment.
A ticket cannot be opened from its address.	Links to a single ticket open the ticket list.	Parse the ticket id in the storefront route.

12 Glossary

The terms used here are defined in section 1.3 of the Software Requirements Specification.